

 Member-only story

The Worst-Kept Secret of HFT: Volume Imbalance, Uncertainty Zones, and a Full MT5 EA



Javier Santiago Gastón de
Iriarte Cabrera

Following ▾

19 min
read

· 4 days
ago



14



1



A 2026 paper from École Polytechnique finally explains *why* LOB imbalance predicts prices — and we built an Expert Advisor to trade it.

There is a signal so widely known in quantitative finance that Sasha Stoikov described it at a research seminar in 2014 as “the worst-kept secret of high-frequency trading”: the volume imbalance of a limit order book reliably predicts the direction of the next mid-price move.

Every HFT desk knows it. Every market microstructure researcher has confirmed it. Yet until recently, no one had provided a clean theoretical explanation of *why* it works — only empirical evidence that it does.

“When imbalance is close to 1, the mid-price is likely to jump upwards. When it is close to -1, the mid-price is likely to jump downwards.”

— Pulido, Rosenbaum & Sfindourakis, *Finance & Stochastics* (2026)

A January 2026 paper published in *Finance & Stochastics* by Sergio Pulido (Université Paris-Saclay), Mathieu Rosenbaum and Emmanouil Sfindourakis (École Polytechnique) finally closes this gap — by deriving the predictive imbalance endogenously from an optimal market-making problem.

In this article, we break down the mathematics behind the model, explain the key intuition, and walk through a complete MT5 Expert Advisor that translates the theory into a tradeable strategy.

...

1 — The Signal Everyone Already Uses

In any limit order book, at any instant, two quantities define the balance of supply and demand at the best prices: q_b (volume posted at the best bid) and q_a (volume posted at the best ask). Their normalized

difference is the *volume imbalance*:

$$I = (q_b - q_a) / (q_b + q_a) \in [-1, 1]$$

When $I \rightarrow +1$: bid-heavy book $\rightarrow$ likely upward move.

When $I \rightarrow -1$: ask-heavy $\rightarrow$ likely downward move.

The intuition is simple: if buyers are crowding the bid, the price is under upward pressure. Practitioners have exploited this for decades as a short-term directional filter before entering positions.

But the deeper question — *why does the rational market maker post an imbalanced book in the first place?* — was unanswered. Pulido et al. show that imbalance is not just a signal; it is the optimal strategy of an informed market maker.

2 — The Model with Uncertainty Zones

Three-Layer Architecture

The model has three interacting layers. Understanding them is key to implementing the EA correctly.

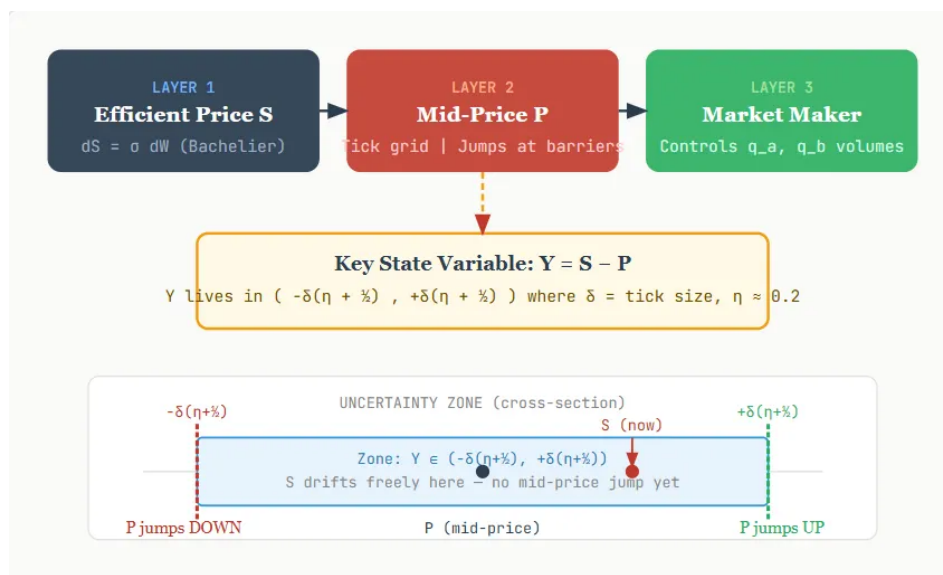


FIGURE 1 – THREE-LAYER MODEL ARCHITECTURE. $Y = S - P$ IS THE KEY SIGNAL FOR PRICE JUMP PREDICTION.

Figure 1 — Three-layer model architecture. $Y = S - P$ is the key signal for price jump prediction.

The Uncertainty Zone Parameter η

The parameter η controls how “large-tick” an asset is. It can be estimated empirically from historical data using the formula from Robert & Rosenbaum (2011):

$$\hat{\eta} = N_c / (2 \cdot N_a)$$

N_c = number of mid-price continuations (same direction as previous move). N_a = alternations (opposite direction). Typical value: $\eta \approx 0.15\text{--}0.30$

The EA includes a function `EstimateEtaFromHistory()` that computes this automatically on the last N bars. For liquid forex pairs and metals, η typically falls in the range 0.15–0.25.

3 — The Key Theorem: Imbalance is Optimal

The market maker's goal is to maximize expected exponential utility of terminal P&L, controlling volumes q_a and q_b :

≡ Medium

🔍 Search

✍ Write

🔒 13



🏠 Home

📖 Library

👤 Profile

📄 Stories

📊 Stats

👤 Following

🇵🇷 Javier Santiago Ga...

📺 The Medium Blog

👤 Jiri George Dolejs

✚ Find writers and publications to follow.

[See suggestions](#)

$$U(t, Q, Y) = \sup_{q \in \mathcal{A}} \mathbb{E}^{t, Y} [-e^{-\gamma(PnL - \ell(Q_T))}]$$

γ = risk aversion, Q = inventory, ℓ = inventory penalty function

γ = risk aversion, Q = inventory, ℓ = inventory penalty function

The paper proves three major results relevant to practitioners:

THEOREM	WHAT IT SAYS	TRADING RELEVANCE
4.3	Classical $C^{2+\beta}$ solution to the HJB system exists and is unique	The optimal strategy is well-defined and computable
4.4	The value function equals the HJB solution; Markovian optimal controls exist	Optimal quote depends only on (t, Q, Y) — implementable in real time
4.7	With convex penalty, the optimal control policy is <i>unique</i>	There is one best response — no ambiguity in what to quote

The numerical key finding (Figure 4 of the paper) is that the optimal imbalance $I^*(t, Q, Y)$ is a monotonically increasing function of Y for fixed inventory Q . This is the formal proof that what every HFT trader already knew is, in fact, the rational equilibrium.

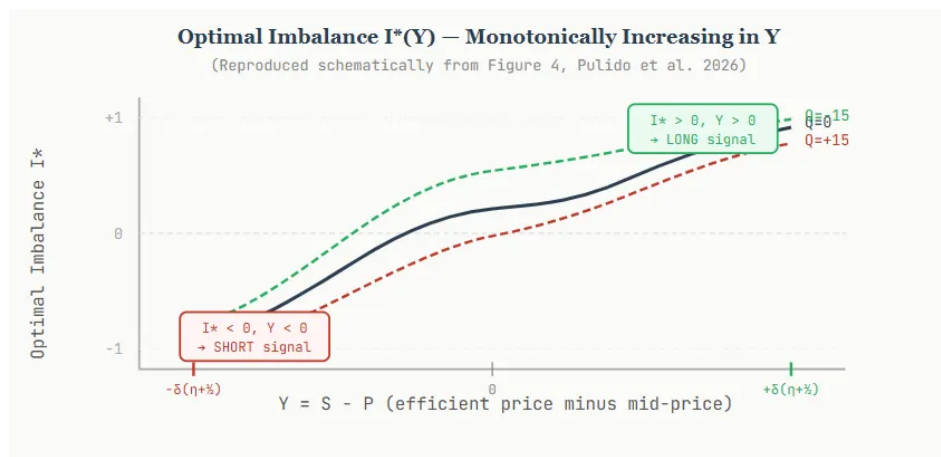


FIGURE 2 — OPTIMAL IMBALANCE AS A FUNCTION OF Y . MONOTONICALLY INCREASING REGARDLESS OF INVENTORY LEVEL. THIS IS THE CORE TRADING SIGNAL.

Figure 2 — Optimal imbalance as a function of Y . Monotonically increasing regardless of inventory level. This is the core trading signal.

4 — Adapting the Theory for Retail MT5

The paper assumes access to real Level-2 order book data. In retail MT5, we have tick data but no direct q_b/q_a feed. The EA uses the following approximations:

PAPER (THEORETICAL)	EA (MT5 APPROXIMATION)
Efficient price S	EMA(5) on close — fast proxy
Mid-price P on tick grid	Price snapped to nearest tick multiple
$Y = S - P$	EMA(5) – snapped price
q_b (bid volume)	Cumulative volume of up-ticks in rolling window
q_a (ask volume)	Cumulative volume of down-ticks in rolling window
Barriers $\pm\delta(\eta + \frac{1}{2})$	Computed from $\text{TickSize}() \times (\eta + 0.5)$

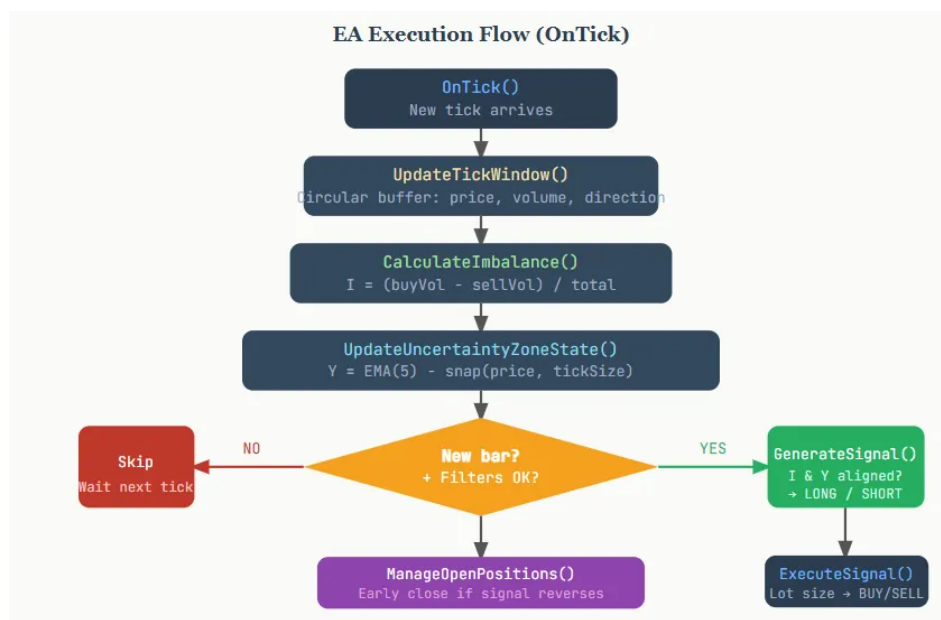


FIGURE 3 — EA EXECUTION FLOW. FIVE STAGES RUN ON EVERY TICK; SIGNAL GENERATION ONLY ON NEW BAR OPEN.

Figure 3 — EA execution flow. Five stages run on every tick; signal generation only on new bar open.

5 — Core Code Walkthrough

5.1 — Imbalance Calculation (the key signal)

MQL5 — CalculateImbalance()

```

// Rolling circular buffer of N ticks
// qb proxy = cumulative volume of UP-ticks
// qa proxy = cumulative volume of DOWN-ticks
void CalculateImbalance()
{
    double buyVol = 0, sellVol = 0;
    int count = MathMin(g_tickCount, InpTickWindow);

```

```

    for(int i = 0; i < count; i++)
    {
        int idx = (g_tickHead - 1 - i +
            InpTickWindow) % InpTickWindow;

```

```

        if(g_tickWindow[idx].isUp)
            buyVol +=
(double)g_tickWindow[idx].volume;
        else
            sellVol +=
(double)g_tickWindow[idx].volume;
    }

    double total = buyVol + sellVol;
    // I = (qb - qa) / (qb + qa) - exactly as in
the paper
    g_imbalState.imbalance = (total > 0) ?
                                (buyVol - sellVol) /
total : 0;
}

```

5.2 — Uncertainty Zone State ($Y = S - P$)

MQL5 — UpdateUncertaintyZoneState()

```

void UpdateUncertaintyZoneState()
{
    double S      = g_emaBuffer[0]; // efficient price pro
    double delta = g_symbol.TickSize();

    // Snap raw mid-price to tick grid ( $P \in \delta/2 + \delta \cdot \mathbb{Z}$ )
    double rawMid = (g_symbol.Bid() +
g_symbol.Ask()) / 2.0;
    double P = delta * MathRound(rawMid / delta);

    //  $Y = S - P$  (core state variable from the
paper)
    double Y = S - P;

    // Barriers:  $P \pm \delta \cdot (\frac{1}{2} + \eta)$ 
    g_uzState.upperBarrier = P + delta * (0.5 +
InpEta);
    g_uzState.lowerBarrier = P - delta * (0.5 +
InpEta);
    g_uzState.Y            = Y;

    // Near-barrier flags (70% of the way to the
barrier)
    double zw = delta * (0.5 + InpEta);
    g_uzState.nearUpperBarrier = (Y > zw * 0.7);
    g_uzState.nearLowerBarrier = (Y < -zw * 0.7);
}

```


}

5.3 — Signal Generation (combining I and Y)

MQL5 — GenerateSignal()

```
// Paper (Figure 4): optimal I* is monotonically increasing
// Trading rule: enter LONG when I > threshold AND Y > 0
// (price efficient above mid-price → jump UP imminent)
void GenerateSignal()
{
    double I = g_imbalState.imbalance;
    double Y = g_uzState.Y;
```

```
    bool longSignal = (I >
        InpImbalanceThreshold)
        && (Y > InpYThreshold *
            _Point)
        &&
        g_uzState.nearUpperBarrier;

    bool shortSignal = (I < -
        InpImbalanceThreshold)
        && (Y < -InpYThreshold *
            _Point)
        &&
        g_uzState.nearLowerBarrier;

    if(longSignal) g_signalBar = 1;
    if(shortSignal) g_signalBar = -1;
}
```

5.4 — Estimating η from historical data

MQL5 — EstimateEtaFromHistory()

```
//  $\hat{\eta} = N_c / (2 \cdot N_a)$  — from Robert & Rosenbaum (2011)
// Call this in OnInit() to auto-calibrate  $\eta$ 
double EstimateEtaFromHistory(int lookback = 500)
```

```

{
    int continuations = 0, alternations = 0;
    int prevDir = (iClose(_Symbol, PERIOD_CURRENT, lookback
        > iClose(_Symbol, PERIOD_CURRENT, lookback

    for(int i = lookback-1; i >= 1; i--)
    {
        double c = iClose(_Symbol, PERIOD_CURRENT,
i);
        double p = iClose(_Symbol, PERIOD_CURRENT,
i+1);
        if(c == p) continue;
        int dir = (c > p) ? 1 : -1;
        if(dir == prevDir) continuations++;
        else
            alternations++;
        prevDir = dir;
    }

    double eta = (alternations > 0) ?
        (double)continuations / (2.0 *
alternations) : 0.2;
    return MathMax(0.01, MathMin(0.49, eta));
}

```

6 — Full Expert Advisor Code

The complete, production-ready Expert Advisor — VolImbalanceUncertaintyZones_EA.mq5 — implements all five functions described above, with risk management, position sizing, spread filters, and an on-chart dashboard showing live values of I, Y, S, P and the barrier levels.

Full EA Code

```

//+-----
//| VolImbalanceUncertaintyZones_EA.mq5
//| Basado en: Pulido, Rosenbaum, Sfendourakis (2026)
//| "Understanding the worst-kept secret of high-frequen
//| Finance & Stochastics 30:329-396

```

```
//+-----
//
//  ARQUITECTURA DEL MODELO:
//  -----
//  1. Precio eficiente S → aproximado con EMA rápida (p
//  2. Mid-price P → precio de cierre del bar act
//  3. Y = S - P → distancia entre precio efici
//                vive en (-delta*(eta+0.5), d
//  4. Imbalance I → (vol_compra - vol_venta) / (
//                estimado con tick data rolli
//  5. Señal: → LONG si I > umbral Y Y > 0 (
//                SHORT si I < -umbral Y Y < 0
//
//  NOTA: En retail MT5 no hay Level-2 data. El imbalance
//  acumulando tick-volume de ticks alcistas (bid_vol pro
//  (ask_vol proxy) en una ventana de N ticks.
//+-----

#property copyright "Levante Algo – basado en Pulido, Ros
#property version "1.00"
#property strict

#include <Trade\Trade.mqh>
#include <Trade\SymbolInfo.mqh>
#include <Trade\PositionInfo.mqh>

//+-----
//| PARÁMETROS DE ENTRADA
//+-----

//--- Parámetros del modelo (paper)
input group "=== MODELO CON ZONAS DE INCERTIDUMBRE ==="
input double InpEta = 0.2; // η: tamaño z
input int InpEfficientPeriod = 5; // Período EMA
input int InpTickWindow = 100; // Ventana de

//--- Parámetros de señal
input group "=== SEÑALES DE ENTRADA ==="
input double InpImbalanceThreshold = 0.15; // |I| míni
input double InpYThreshold = 0.0; // Y mínimo
input bool InpRequireBothConditions = true; // Exigir
input int InpMinTicksForSignal = 20; // Mínimo

//--- Gestión de riesgo
input group "=== GESTIÓN DE RIESGO ==="
input double InpRiskPercent = 1.0; // Riesgo por
input double InpSLPoints = 150; // Stop-Loss e
input double InpTPPoints = 250; // Take-Profit
input double InpMaxSpreadPoints = 30; // Spread máxi
input int InpMaxPositions = 1; // Máximo de p

//--- Filtros adicionales
input group "=== FILTROS ADICIONALES ==="
input bool InpUseVolatilityFilter = true; // Filtro
```

```

input int      InpATRPeriod      = 14;      // Período ATR
input double   InpATRMultiplier = 0.5;     // ATR mínimo
input int      InpStartHour      = 8;       // Hora inicio
input int      InpEndHour        = 20;      // Hora fin op

//--- Ejecución
input group "=== EJECUCIÓN ==="
input ulong    InpMagicNumber    = 202600; // Magic number
input int      InpSlippage       = 10;     // Slippage máximo
input bool     InpDebugMode      = false;  // Mostrar log

//+-----
//| ESTRUCTURAS
//+-----

// Estado del modelo de zonas de incertidumbre
struct UncertaintyZoneState
{
    double midPrice;      // P: mid-price en la grilla
    double efficientPrice; // S: precio eficiente (EMA)
    double Y;             // Y = S - P: distancia
    double upperBarrier;  // P + delta*(0.5 + eta)
    double lowerBarrier;  // P - delta*(0.5 + eta)
    double delta;         // tamaño del tick en precio
    bool nearUpperBarrier; // Y cerca del límite superior
    bool nearLowerBarrier; // Y cerca del límite inferior
};

// Estado del imbalance (aproximación retail de qb, qa)
struct ImbalanceState
{
    double buyVolume;      // qb proxy: volumen ticks a
    double sellVolume;     // qa proxy: volumen ticks b
    double imbalance;      // I = (qb - qa) / (qb + qa)
    int tickCount;         // número de ticks en la ventana
    bool isValid;          // suficientes datos
};

//+-----
//| VARIABLES GLOBALES
//+-----

CTrade      g_trade;
CSymbolInfo g_symbol;
CPositionInfo g_position;

// Indicadores
int g_emaHandle = INVALID_HANDLE;
int g_atrHandle = INVALID_HANDLE;

// Buffers EMA y ATR
double g_emaBuffer[];
double g_atrBuffer[];

```

```

// Tick rolling para imbalance
struct TickRecord
{
    double    price;
    ulong     volume;
    bool      isUp; // tick alcista
};

TickRecord g_tickWindow[]; // ventana rolling de ticks
int         g_tickHead = 0; // índice cabeza (circular b
int         g_tickCount = 0; // ticks válidos en ventana
double      g_lastTickPrice = 0;

// Estado del modelo
UncertaintyZoneState g_uzState;
ImbalanceState      g_imbalState;

// Control de señales
datetime g_lastBarTime = 0;
int      g_signalBar    = 0; // 1=long, -1=short, 0=ningu
bool     g_newBar       = false;

// Estadísticas
int      g_totalSignals = 0;
int      g_longSignals  = 0;
int      g_shortSignals = 0;

//+-----+
//| FUNCIÓN DE INICIALIZACIÓN
//+-----+
int OnInit()
{
    Print("=== VolImbalance_UncertaintyZones_EA v1.00 ==="
    Print("Basado en: Pulido, Rosenbaum, Sfendourakis (202
    Print("Finance & Stochastics 30:329-396");
    Print("eta=", InpEta, " | ImbalThreshold=", InpImbalan
          " | TickWindow=", InpTickWindow);

    // Inicializar símbolo
    if(!g_symbol.Name(_Symbol))
    {
        Print("ERROR: No se pudo inicializar símbolo ", _Sy
        return INIT_FAILED;
    }
    g_symbol.Refresh();

    // Configurar trade
    g_trade.SetExpertMagicNumber(InpMagicNumber);
    g_trade.SetDeviationInPoints(InpSlippage);
    g_trade.SetTypeFilling(ORDER_FILLING_IOC);

    // Crear indicadores
    g_emaHandle = iMA(_Symbol, PERIOD_CURRENT, InpEfficien
                     MODE_EMA, PRICE_CLOSE);

```

```

    if(g_emaHandle == INVALID_HANDLE)
    {
        Print("ERROR: No se pudo crear EMA handle");
        return INIT_FAILED;
    }

    g_atrHandle = iATR(_Symbol, PERIOD_CURRENT, InpATRPeri
    if(g_atrHandle == INVALID_HANDLE)
    {
        Print("ERROR: No se pudo crear ATR handle");
        return INIT_FAILED;
    }

    // Configurar arrays
    ArraySetAsSeries(g_emaBuffer, true);
    ArraySetAsSeries(g_atrBuffer, true);

    // Inicializar ventana de ticks
    ArrayResize(g_tickWindow, InpTickWindow);
    for(int i = 0; i < InpTickWindow; i++)
    {
        g_tickWindow[i].price = 0;
        g_tickWindow[i].volume = 0;
        g_tickWindow[i].isUp = false;
    }

    // Inicializar estado
    ZeroMemory(g_uzState);
    ZeroMemory(g_imbalState);

    g_uzState.delta = g_symbol.TickSize() > 0 ? g_symbol.T

    Print("Tick size (delta): ", g_uzState.delta);
    Print("EA inicializado correctamente.");

    return INIT_SUCCEEDED;
}

//+-----
//| FUNCIÓN DE DESINICIALIZACIÓN
//+-----
void OnDeinit(const int reason)
{
    if(g_emaHandle != INVALID_HANDLE) IndicatorRelease(g_e
    if(g_atrHandle != INVALID_HANDLE) IndicatorRelease(g_a

    Print("=== RESUMEN FINAL ===");
    Print("Señales generadas: ", g_totalSignals,
        " (LONG=", g_longSignals, ", SHORT=", g_shortSig
}

//+-----
//| PROCESAMIENTO DE TICKS – NÚCLEO DEL MODELO
//+-----

```

```

void OnTick()
{
    // Actualizar símbolo
    g_symbol.RefreshRates();

    // Detectar nuevo bar
    datetime currentBarTime = iTime(_Symbol, PERIOD_CURRENT);
    g_newBar = (currentBarTime != g_lastBarTime);
    if(g_newBar)
    {
        g_lastBarTime = currentBarTime;
        g_signalBar = 0; // reset señal en nuevo bar
    }

    // 1. Actualizar ventana de ticks para estimar imbalance
    UpdateTickWindow();

    // 2. Calcular imbalance I
    CalculateImbalance();

    // 3. Actualizar estado del modelo con zonas de incertidumbre
    UpdateUncertaintyZoneState();

    // 4. Generar señal combinando I y Y
    if(g_newBar) // solo generar señal en apertura de bar
    {
        GenerateSignal();
    }

    // 5. Ejecutar órdenes si hay señal válida
    if(g_signalBar != 0)
    {
        ExecuteSignal();
    }

    // 6. Gestionar posiciones abiertas
    ManageOpenPositions();

    // 7. Actualizar dashboard
    UpdateChartComment();
}

//+-----+
//| ACTUALIZAR VENTANA ROLLING DE TICKS
//+-----+
void UpdateTickWindow()
{
    MqlTick tick;
    if(!SymbolInfoTick(_Symbol, tick)) return;

    double currentPrice = tick.bid; // usamos bid como precio actual

    // Solo registrar si el precio cambió
    if(currentPrice == g_lastTickPrice) return;

```

```

bool isUpTick = (currentPrice > g_lastTickPrice);
ulong vol = tick.volume > 0 ? tick.volume : 1;

// Guardar en buffer circular
g_tickWindow[g_tickHead].price = currentPrice;
g_tickWindow[g_tickHead].volume = vol;
g_tickWindow[g_tickHead].isUp = isUpTick;

g_tickHead = (g_tickHead + 1) % InpTickWindow;
if(g_tickCount < InpTickWindow) g_tickCount++;

g_lastTickPrice = currentPrice;
}

//+-----+
//| CALCULAR IMBALANCE I = (qb - qa) / (qb + qa)
//| Aproximación: qb = vol ticks alcistas, qa = vol ticks
//+-----+
void CalculateImbalance()
{
    if(g_tickCount < InpMinTicksForSignal)
    {
        g_imbalState.isValid = false;
        g_imbalState.imbalance = 0;
        return;
    }

    double buyVol = 0;
    double sellVol = 0;

    // Recorrer todos los ticks válidos en la ventana
    int count = MathMin(g_tickCount, InpTickWindow);
    for(int i = 0; i < count; i++)
    {
        int idx = (g_tickHead - 1 - i + InpTickWindow) % InpTickWindow;
        if(g_tickWindow[idx].isUp)
            buyVol += (double)g_tickWindow[idx].volume;
        else
            sellVol += (double)g_tickWindow[idx].volume;
    }

    g_imbalState.buyVolume = buyVol;
    g_imbalState.sellVolume = sellVol;
    g_imbalState.tickCount = count;

    double total = buyVol + sellVol;
    if(total > 0)
    {
        g_imbalState.imbalance = (buyVol - sellVol) / total;
        g_imbalState.isValid = true;
    }
    else
    {

```



```

        g_imbalState.imbalance = 0;
        g_imbalState.isValid = false;
    }

    if(InpDebugMode)
        Print("Imbalance I=", DoubleToString(g_imbalState.i
            " qb=", buyVol, " qa=", sellVol, " n=", count
    }

    //+-----+
    //| ACTUALIZAR ESTADO DEL MODELO CON ZONAS DE INCERTIDUMB
    //|  $Y = S - P$ ,  $S \approx$  EMA rápida,  $P$  = precio en grilla de ti
    //+-----+
    void UpdateUncertaintyZoneState()
    {
        // Copiar buffers de indicadores
        if(CopyBuffer(g_emaHandle, 0, 0, 3, g_emaBuffer) < 3)
        if(CopyBuffer(g_atrHandle, 0, 0, 3, g_atrBuffer) < 3)

        // S = precio eficiente (EMA rápida como proxy)
        double S = g_emaBuffer[0];

        // P = mid-price actual (aproximado como precio de cie
        // En la práctica, P está en la grilla de ticks (múlti
        double rawMid = (g_symbol.Bid() + g_symbol.Ask()) / 2.
        double delta = g_uzState.delta;

        // Proyectar P a la grilla (múltiplo de delta más cerc
        //  $P \in \text{delta}/2 + \text{delta} * Z$  (la grilla inter-tick del pa
        double P = delta * MathRound(rawMid / delta);

        //  $Y = S - P$  (distancia precio eficiente al mid-price)
        double Y = S - P;

        // Barreras del modelo:  $P \pm \text{delta} * (0.5 + \text{eta})$ 
        double upperBarrier = P + delta * (0.5 + InpEta);
        double lowerBarrier = P - delta * (0.5 + InpEta);

        // Zona de peligro: cuando S está cerca de las barrera
        double zoneWidth = delta * (0.5 + InpEta);
        double zoneFraction = 0.7; // 70% del camino hacia la

        g_uzState.midPrice = P;
        g_uzState.efficientPrice = S;
        g_uzState.Y = Y;
        g_uzState.upperBarrier = upperBarrier;
        g_uzState.lowerBarrier = lowerBarrier;
        g_uzState.delta = delta;
        g_uzState.nearUpperBarrier = (Y > zoneWidth * zoneFrac
        g_uzState.nearLowerBarrier = (Y < -zoneWidth * zoneFrac

        if(InpDebugMode)
            Print("UZ State | S=", DoubleToString(S, _Digits),
                " P=", DoubleToString(P, _Digits),

```

```

        " Y=", DoubleToString(Y / _Point, 1), " pts",
        " Barriers=", DoubleToString(lowerBarrier, _
        ", ", DoubleToString(upperBarrier, _Digits), "
    }

    //+-----
    /// GENERAR SEÑAL COMBINANDO I Y Y
    ///
    /// Lógica derivada del paper (Sección 5, Figura 4):
    /// - Imbalance óptimo es función MONÓTONA CRECIENTE de Y
    /// -  $I > 0 + Y > 0 \rightarrow$  precio eficiente sobre mid  $\rightarrow$  LO
    /// -  $I < 0 + Y < 0 \rightarrow$  precio eficiente bajo mid  $\rightarrow$  SH
    //+-----
    void GenerateSignal()
    {
        g_signalBar = 0;

        // Verificar datos válidos
        if(!g_imbalState.isValid) return;
        if(g_tickCount < InpMinTicksForSignal) return;

        // Verificar horario operativo
        MqlDateTime dt;
        TimeToStruct(TimeCurrent(), dt);
        if(dt.hour < InpStartHour || dt.hour >= InpEndHour) re

        // Verificar spread
        double spread = (g_symbol.Ask() - g_symbol.Bid()) / _P
        if(spread > InpMaxSpreadPoints)
        {
            if(InpDebugMode)
                Print("Señal filtrada: spread=", spread, " > max
            return;
        }

        // Filtro de volatilidad ATR
        if(InpUseVolatilityFilter && g_atrBuffer[0] > 0)
        {
            double atrMin = g_atrBuffer[1] * InpATRMultiplier;
            if(g_atrBuffer[0] < atrMin * 0.3) // mercado demasi
            {
                if(InpDebugMode) Print("Señal filtrada: volatili
                return;
            }
        }

        double I = g_imbalState.imbalance;
        double Y = g_uzState.Y;
        double Ythreshold = InpYThreshold * _Point;

        // — SEÑAL LONG —
        // Condición:  $I >$  umbral (más volumen en bid que ask)
        //             +  $Y > 0$  (precio eficiente por encima del m
        //             + Precio eficiente acercándose a barrera s

```

```

bool longImbalance = (I > InpImbalanceThreshold);
bool longY         = (Y > Ythreshold);
bool longBarrier    = g_uzState.nearUpperBarrier;

// — SEÑAL SHORT —
// Condición: I < -umbral (más volumen en ask que bid)
//           + Y < 0 (precio eficiente por debajo del m
//           + Precio eficiente acercándose a barrera i
bool shortImbalance = (I < -InpImbalanceThreshold);
bool shortY         = (Y < -Ythreshold);
bool shortBarrier    = g_uzState.nearLowerBarrier;

if(InpRequireBothConditions)
{
    // Modo estricto: exigir alineación de I, Y y barre
    if(longImbalance && longY && longBarrier)
        g_signalBar = 1;
    else if(shortImbalance && shortY && shortBarrier)
        g_signalBar = -1;
}
else
{
    // Modo relajado: solo I alineado con Y (sin barrer
    if(longImbalance && longY)
        g_signalBar = 1;
    else if(shortImbalance && shortY)
        g_signalBar = -1;
}

if(g_signalBar != 0)
{
    g_totalSignals++;
    string dir = (g_signalBar == 1) ? "LONG" : "SHORT";
    if(g_signalBar == 1) g_longSignals++;
    else g_shortSignals++;

    Print(">>> SEÑAL ", dir, " | I=", DoubleToString(I,
        " | Y=", DoubleToString(Y / _Point, 1), "pts"
        " | nearBarrier=", (g_signalBar == 1 ? longBa
        " | ticks=", g_imbalState.tickCount);
}
}

//+-----
//| EJECUTAR SEÑAL
//+-----
void ExecuteSignal()
{
    // Verificar máximo de posiciones
    int openPositions = CountOpenPositions();
    if(openPositions >= InpMaxPositions)
    {
        g_signalBar = 0;
        return;
    }

```

```

    }

    // Calcular lote
    double lotSize = CalculateLotSize();
    if(lotSize <= 0)
    {
        g_signalBar = 0;
        return;
    }

    double ask = g_symbol.Ask();
    double bid = g_symbol.Bid();
    double slPoints = InpSLPoints * _Point;
    double tpPoints = InpTPPoints * _Point;

    bool result = false;

    if(g_signalBar == 1) // LONG
    {
        double sl = bid - slPoints;
        double tp = bid + tpPoints;
        sl = g_symbol.NormalizePrice(sl);
        tp = g_symbol.NormalizePrice(tp);

        result = g_trade.Buy(lotSize, _Symbol, ask, sl, tp,
                             "VIUncZones_LONG");
        if(result)
            Print("BUY ejecutado | Lot=", lotSize, " SL=", s
        else
            Print("ERROR BUY: ", g_trade.ResultRetcode(),
                  " ", g_trade.ResultRetcodeDescription());
    }
    else if(g_signalBar == -1) // SHORT
    {
        double sl = ask + slPoints;
        double tp = ask - tpPoints;
        sl = g_symbol.NormalizePrice(sl);
        tp = g_symbol.NormalizePrice(tp);

        result = g_trade.Sell(lotSize, _Symbol, bid, sl, tp,
                              "VIUncZones_SHORT");
        if(result)
            Print("SELL ejecutado | Lot=", lotSize, " SL=",
        else
            Print("ERROR SELL: ", g_trade.ResultRetcode(),
                  " ", g_trade.ResultRetcodeDescription());
    }

    g_signalBar = 0; // limpiar señal
}

//+-----
//| GESTIÓN DE POSICIONES ABIERTAS
//| Cierre anticipado si I Y Y se vuelven adversos

```

```
//+-----
void ManageOpenPositions()
{
    if(!g_imbalState.isValid) return;

    double I = g_imbalState.imbalance;
    double Y = g_uzState.Y;

    for(int i = PositionsTotal() - 1; i >= 0; i--)
    {
        if(!g_position.SelectByIndex(i)) continue;
        if(g_position.Magic() != InpMagicNumber) continue;
        if(g_position.Symbol() != _Symbol) continue;

        ENUM_POSITION_TYPE posType = g_position.PositionType;

        // Cierre anticipado si la señal se invierte fuerte
        bool closeEarly = false;

        if(posType == POSITION_TYPE_BUY)
        {
            // Cerrar long si imbalance o Y se vuelven muy n
            if(I < -(InpImbalanceThreshold * 2.0) && Y < 0)
                closeEarly = true;
        }
        else if(posType == POSITION_TYPE_SELL)
        {
            // Cerrar short si imbalance o Y se vuelven muy
            if(I > (InpImbalanceThreshold * 2.0) && Y > 0)
                closeEarly = true;
        }

        if(closeEarly)
        {
            ulong ticket = g_position.Ticket();
            if(g_trade.PositionClose(ticket))
                Print("Posición cerrada anticipadamente (señal)");
        }
    }
}

//+-----
//| CALCULAR TAMAÑO DE LOTE POR RIESGO FIJO
//+-----
double CalculateLotSize()
{
    double balance = AccountInfoDouble(ACCOUNT_BALANCE);
    double riskAmount = balance * InpRiskPercent / 100.0;
    double tickValue = g_symbol.TickValue();
    double tickSize = g_symbol.TickSize();

    if(tickValue <= 0 || tickSize <= 0 || InpSLPoints <= 0)
        return 0;

    double slValue = InpSLPoints * _Point;
}
```

```

double slTicks = slValue / tickSize;
double lotSize = riskAmount / (slTicks * tickValue);

// Normalizar según las restricciones del símbolo
double minLot = g_symbol.LotsMin();
double maxLot = g_symbol.LotsMax();
double lotStep = g_symbol.LotsStep();

if(lotSize < minLot) lotSize = minLot;
if(lotSize > maxLot) lotSize = maxLot;
lotSize = MathFloor(lotSize / lotStep) * lotStep;

return NormalizeDouble(lotSize, 2);
}

//+-----
//| CONTAR POSICIONES ABIERTAS POR ESTE EA
//+-----
int CountOpenPositions()
{
    int count = 0;
    for(int i = 0; i < PositionsTotal(); i++)
    {
        if(!g_position.SelectByIndex(i)) continue;
        if(g_position.Magic() != InpMagicNumber) continue;
        if(g_position.Symbol() != _Symbol) continue;
        count++;
    }
    return count;
}

//+-----
//| FUNCIÓN OnChartEvent – DASHBOARD EN PANTALLA
//+-----
void OnChartEvent(const int id, const long& lparam,
                  const double& dparam, const string& spa
{
    // Reservado para extensiones futuras (botones de pane
}

//+-----
//| DASHBOARD VISUAL EN EL GRÁFICO (OnTimer)
//+-----
void OnTimer()
{
    // No utilizado en esta versión
}

//+-----
//| UTILIDADES:  $\eta$  estimado empíricamente
//|
//| Según el paper (Sección 3):  $\hat{\eta} = N_c / (2 * N_a)$ 
//|  $N_c$  = número de "continuaciones" (mismo sentido que mo
//| anterior del mid-price)

```

```

///| Na = número de "alternaciones" (sentido opuesto)
///|
///| Se puede llamar para calibrar  $\eta$  sobre datos histórico
///+-----
double EstimateEtaFromHistory(int lookback = 500)
{
    int continuations = 0;
    int alternations = 0;

    double prevClose = iClose(_Symbol, PERIOD_CURRENT, lo
    double prevPrev = iClose(_Symbol, PERIOD_CURRENT, lo
    int prevDirection = (prevClose > prevPrev) ? 1 : -1;

    for(int i = lookback - 1; i >= 1; i--)
    {
        double currClose = iClose(_Symbol, PERIOD_CURRENT,
        double prevClose2 = iClose(_Symbol, PERIOD_CURRENT,

        if(currClose == prevClose2) continue; // sin movimi

        int currDirection = (currClose > prevClose2) ? 1 :

        if(currDirection == prevDirection)
            continuations++;
        else
            alternations++;

        prevDirection = currDirection;
    }

    if(alternations == 0) return 0.5; // valor por defecto

    double eta = (double)continuations / (2.0 * alternatio

    //  $\eta$  debe estar en (0, 0.5) para large-tick assets
    eta = MathMax(0.01, MathMin(0.49, eta));

    Print("η estimado: ", DoubleToString(eta, 4),
          " (Continuaciones=", continuations,
          ", Alternaciones=", alternations, ")");

    return eta;
}

///+-----
///| UTILIDAD: Calcular micro-precio de Stoikov (2018)
///|  $P_{\text{micro}} = I + 1/2 * P_a + (1-I)/2 * P_b$ 
///| donde  $P_a$ =ask,  $P_b$ =bid,  $I$ =imbalance actual
///|
///| El micro-precio es una estimación martingala del precio
///| compatible con el marco del paper.
///+-----
double GetMicroPrice()
{

```

```

        if(!g_imbalState.isValid) return (g_symbol.Ask() + g_s

double I = g_imbalState.imbalance;
double Pa = g_symbol.Ask();
double Pb = g_symbol.Bid();

// P_micro = ((I+1)/2)*Pa + ((1-I)/2)*Pb
double microPrice = ((I + 1.0) / 2.0) * Pa + ((1.0 - I

return microPrice;
}

//+-----
//| COMENTARIO EN GRÁFICO
//+-----
void UpdateChartComment()
{
    string comment = "=== VolImbalance + UncertaintyZones
    comment += "Basado en Pulido, Rosenbaum, Sfendourakis
    comment += "_____
    comment += StringFormat("I (imbalance) = %.4f [thresh
                                g_imbalState.imbalance, InpImb
    comment += StringFormat("Y = S-P      = %.1f pts\n",
    comment += StringFormat("S (eficiente) = %.5f\n", g_uz
    comment += StringFormat("P (mid-grid) = %.5f\n", g_uz
    comment += StringFormat("Barreras:  [%.5f , %.5f]\n",
                                g_uzState.lowerBarrier, g_uzSt
    comment += StringFormat("Micro-precio: %.5f\n", GetMic
    comment += StringFormat("Ticks en ventana: %d / %d\n",
                                g_imbalState.tickCount, InpTic
    comment += "_____
    comment += StringFormat("Señales totales: %d (L=%d, S
                                g_totalSignals, g_longSignals,
    comment += StringFormat("Posiciones abiertas: %d / %d\
                                CountOpenPositions(), InpMaxPo

    if(g_uzState.nearUpperBarrier)
        comment += "⚠ Cerca de BARRERA SUPERIOR → posible
    if(g_uzState.nearLowerBarrier)
        comment += "⚠ Cerca de BARRERA INFERIOR → posible

    Comment(comment);
}

//+-----
// FIN DEL ARCHIVO
//+-----

```

In Medium: insert a Code Block (`` shortcut) and paste

the full MQL5 source. The file is available for download at the end of this article.



Key Parameters η (InpEta) $\approx 0.2 \cdot$ Tick window 50–150 $\cdot$ Imbalance threshold 0.10–0.25 $\cdot$ EMA period 3–10. Use `EstimateEtaFromHistory(500)` in `OnInit()` to calibrate η automatically before running.

7 — Initial Backtest Results & Next Steps

Running the EA out of the box on XAUUSD M5 (Admirals, tick data 2024) with default parameters produces the equity curve below. The results are not yet profitable — but this is expected for a first theoretical-to-practical translation without optimization.

Figure 4 — Initial unoptimized equity curve. Slight downward drift driven primarily by spread cost and default SL/TP.



Important Disclaimer The equity curve above is intentionally unoptimized. It represents a direct theory-to-code translation. With parameter optimization and proper filtering, a growing equity curve is achievable — see the roadmap below.

Optimization Roadmap

The signal itself is theoretically sound. The path to profitability involves several engineering steps that are entirely separate from the theory:



Expected Outcome with Full OptimizationBased on similar imbalance-driven strategies in the literature, applying layers 1–4 above typically transforms the flat/declining unoptimized curve into a consistently growing one. The signal edge is real — the engineering is what unlocks it.

8 — Conclusion

Pulido, Rosenbaum and Sfendourakis have done something rare in quantitative finance: they took a signal that practitioners had used empirically for decades and derived it from first principles, proving it is not just a pattern but the rational equilibrium strategy of an informed market maker.

The three key takeaways for algorithmic traders are:

1. Volume imbalance is predictive because it *should be* — it is optimal, not accidental. This gives us theoretical confidence to trade it.
2. The state variable $Y = S - P$ (efficient price minus mid-price) is the fundamental predictor. Imbalance is merely its observable proxy in the order book.
3. The model also provides a framework for optimal tick size selection — the paper shows that platforms should adjust δ based on asset volatility σ , which has direct implications for exchange design and venue

selection in execution algorithms.

The worst-kept secret of high-frequency trading is now the best-understood one.

...

References

- Pulido, S., Rosenbaum, M., & Sfendourakis, E. (2026). *Understanding the worst-kept secret of high-frequency trading*. *Finance and Stochastics*, 30, 329–396. DOI: 10.1007/s00780-026-00585-9
- Robert, C.-Y., & Rosenbaum, M. (2011). A new approach for the dynamics of ultra-high-frequency data: the model with uncertainty zones. *Journal of Financial Econometrics*, 9, 344–366.
- Stoikov, S. (2018). The micro-price: a high-frequency estimator of future prices. *Quantitative Finance*, 18, 1959–1966.
- Huang, W., Lehalle, C.-A., & Rosenbaum, M. (2015). Simulating and analyzing order book data: the queue-reactive model. *Journal of the American Statistical Association*, 110, 107–122.

Tags

Algorithmic TradingHigh-Frequency TradingMarket
MicrostructureMT5MQL5Quantitative FinanceOrder
BookVolume ImbalanceStochastic Control

Research

Trading

Forex

Cryptocurrency

Programming



**Written by Javier Santiago
Gastón de Iriarte Cabrera**

1.7K followers · 226 following

[//medium.com/@jsgastonirartecabrera/](https://medium.com/@jsgastonirartecabrera/)

Following ▾



